

Pensamiento computacional
Proyecto final plataforma de gestión de eventos

Alejandra Ibarra Ortiz
Miguel Ángel Montoya Rodríguez

Programación II

Carolina Londoño Idarraga

Ingeniería de Sistemas y Computación
Universidad del Quindío

07 mayo 2026

Introducción

Para el desarrollo de este proyecto final es importante realizar primero un análisis del problema y de las necesidades del sistema antes de comenzar la etapa de codificación. El pensamiento computacional permite organizar las ideas, identificar la información importante y dividir el problema en partes más pequeñas para facilitar el diseño de la solución.

En este caso, el proyecto consiste en una plataforma de gestión de eventos y venta de entradas, donde los usuarios podrán consultar eventos, seleccionar asientos, realizar compras y efectuar pagos, mientras que los administradores podrán gestionar eventos, recintos, usuarios y métricas del sistema.

El objetivo de este documento es analizar la estructura general de la aplicación, identificar sus funcionalidades principales y establecer cómo se organizarán las clases y responsabilidades dentro del sistema.

Pensamiento computacional RF-043

Abstracción:

- **¿Qué se solicita finalmente?**

Se solicita desarrollar una plataforma de gestión de eventos y venta de entradas que permita administrar tanto las funciones del usuario como las del administrador.

El sistema debe permitir que los usuarios consulten eventos, seleccionen entradas, agreguen servicios adicionales, realicen pagos y consulten el historial de compras. Además, los administradores podrán gestionar eventos, usuarios, recintos, zonas, asientos e incidencias.

- **¿Qué información es relevante?**

Área del sistema	Información relevante
Gestión de usuarios.	Usuarios, estados de compra.
Gestión de eventos.	Eventos, estados de eventos, incidencias.
Gestión de recintos	Recintos, zonas, asientos.
Gestión de ventas	Compras, entradas, pagos, tarifas.
Servicios complementarios	Servicios adicionales.

También es importante un correcto manejo de la disponibilidad de los asientos, los métodos de pago y los reportes del sistema.

- **¿Cómo se agrupa la información relevante? (Clases y estructura)**

La información se organizará mediante clases que representen las entidades principales del sistema. Cada clase almacenará los atributos y comportamientos necesarios para manejar la información de manera ordenada y eficiente.

Además, las clases estarán relacionadas entre sí para formar la estructura general del sistema. Por ejemplo, un usuario podrá realizar compras, las compras estarán asociadas a entradas, y las entradas a eventos y asientos específicos.

Esta estructura permitirá mantener la información organizada, facilitando la administración de datos y mejorando el funcionamiento general del sistema.

Clases	
Usuario Contendrá la información personal del usuario y sus compras realizadas.	Pago Controlará el procesamiento del pago y el método utilizado.
Evento Representará cada evento disponible dentro de la plataforma.	Tarifa Permitirá definir los precios de las entradas según la zona, tipo de evento o beneficios aplicados.
Recinto Permitirá almacenar la información del lugar donde se realizará el evento.	Incidencia Registrará problemas, reportes o situaciones ocurridas durante un evento o proceso de compra.
Zona Representará las diferentes zonas del recinto, como VIP, general o preferencial.	ServicioAdicional Representará servicios extra ofrecidos al usuario, como parqueadero, comida o acceso preferencial.
Asiento Administrará la disponibilidad y estado de cada asiento.	EstadoCompra Controlará el estado actual de una compra como: pendiente, pagada o cancelada.

Compra Gestionará el proceso de compra realizado por el usuario.	EstadoEvento Permitirá administrar el estado de los eventos como: programado, aplazado, cancelado o finalizado.
Entrada Representará cada ticket generado después del pago.	

- **¿Qué funcionalidades se solicitan finalmente?**

Funcionalidades del usuario:

- Registrarse e iniciar sesión.
- Consultar eventos.
- Filtrar eventos.
- Seleccionar zonas y asientos.
- Realizar compras.
- Agregar servicios adicionales.
- Pagar entradas.
- Consultar historial de compras.
- Descargar comprobantes y reportes.

Funcionalidades del administrador

- Gestionar usuarios.
- Gestionar eventos.
- Gestionar recintos.
- Gestionar zonas y asientos.
- Controlar disponibilidad.
- Gestionar incidencias.
- Consultar métricas y reportes.

Descomposición:

- **¿Cómo se distribuyen las funcionalidades?**

Las funcionalidades del sistema se distribuirán entre diferentes clases, donde cada una tendrá una responsabilidad específica. Esto permite mantener una mejor organización, facilitar el mantenimiento del sistema y evitar que una sola clase controle demasiadas funciones.

La clase **Usuario** se encargará del almacenamiento de la información de los usuarios y permitir acciones como registrarse, iniciar sesión y consultar sus compras.

La clase **Evento** administrará toda la información relacionada con los eventos, como nombre, fecha, horario, disponibilidad y estado del evento.

Las clases **Recinto**, **Zona** y **Asiento** manejarán la estructura física de cada evento.

- **Recinto** representará el lugar donde se realiza el evento.
- **Zona** dividirá el recinto en secciones.
- **Asiento** controlará la disponibilidad y ubicación de cada puesto.

La clase **Compra** gestionará el proceso de compra de entradas, relacionando al usuario con los asientos seleccionados y el total a pagar.

La clase **Pago** administrará los métodos de pago y verificará el estado de las transacciones realizadas.

La clase **Entrada** representará los tickets generados después de una compra exitosa, almacenando información como el evento, asientos y código de acceso.

Además, clases como **Tarifa**, **Incidencia**, **ServicioAdicional**, **EstadoCompra** y **EstadoEvento** ayudarán a complementar el funcionamiento del sistema, permitiendo manejar precios, reportes, servicios extra y estados de control.

● ¿Qué debo hacer para probar las funcionalidades?

Para verificar el funcionamiento del sistema será necesario crear datos de prueba, como usuarios, eventos, recintos y compras simuladas.

También se deben realizar pruebas para validar:

- Iniciar sesión.
- Registro de usuarios.
- Compra de entradas.
- Cambio de estados.
- Disponibilidad de asientos.
- Cancelaciones.
- Generación de reportes.
- Procesamiento de pagos.

Reconocimiento de patrones:

- **¿Qué puedo reutilizar?**

Para el desarrollo del proyecto se pueden reutilizar varias estructuras y conceptos trabajados anteriormente en clase y en otros ejercicios realizados durante el semestre.

- Interfaces.
- Clases abstractas.
- Validaciones.
- Listas.
- Principios SOLID.
- Patrones de diseño:
 - Creacionales.
 - Estructurales.
 - Comportamentales.
- Métodos reutilizables.

Esto permite optimizar el desarrollo, mantener una estructura más ordenada y evitar repetir código innecesariamente.

Codificación:

- **¿Cómo pruebo/escribo la solución en Java?**

La solución será desarrollada en Java utilizando programación orientada a objetos para mantener una mejor organización del proyecto. El sistema se dividirá en clases según la responsabilidad de cada una, permitiendo que el código sea más claro, reutilizable y fácil de mantener.

Inicialmente se implementará la lógica principal del sistema, como la gestión de usuarios, eventos, compras, pagos y disponibilidad de asientos. Después de validar el correcto funcionamiento de estas funcionalidades, se integrará la interfaz gráfica utilizando JavaFX, tal como lo solicita el proyecto.

Para el desarrollo también se aplicarán principios SOLID y diferentes patrones de diseño con el fin de mejorar la estructura del sistema y evitar código repetido o dependencias innecesarias.

Las pruebas se realizarán utilizando datos simulados, verificando procesos como:

- Registro e inicio de sesión.
- Creación de eventos.

- Selección de entradas.
- Procesamiento de pagos.
- Cambios de estado.
- Cancelaciones.
- Generación de reportes.

Además, el proyecto será gestionado mediante Git para llevar control de versiones y facilitar el trabajo colaborativo entre los integrantes del grupo.

Conclusión

El pensamiento computacional permitió analizar el problema antes de comenzar la implementación del sistema. Gracias a esto fue posible identificar las entidades principales, las funcionalidades necesarias y la forma en que se distribuirán las responsabilidades dentro de la aplicación.

Realizar esta etapa de planificación ayuda a reducir errores durante el desarrollo y facilita la construcción de un sistema más organizado, mantenible y escalable.